A Unified Projection Framework for Tensor-Structured Dictionary Learning

Six dictionaries, one harness — and an efficiency ranking system

Sahaj Chhetri

August 17, 2026

Motivation

Flattening a patch discards what made it compressible

Classical DL models a signal as $y \approx Dx$ with unit-norm atoms and a sparse code. For a multiway patch the standard recipe flattens it to a vector of length $n = \prod_i n_i$. Two costs follow:

- Spatial neighbourhoods and spectral smoothness — the redundancy that makes the data compressible — are discarded.
- The dictionary carries $n \cdot K$ free numbers, growing *multiplicatively* with tensor order.

More parameters than the data needs invites over-fitting: the extra capacity is spent modelling noise.

25,920

free parameters — vectorized dictionary, $6 \times 6 \times 6$, $K = 120$

2,160

the same 120 atoms under CP — 12× smaller

126

Tucker mode factors covering 343 implicit atoms — 206×

Constrain the atom, not the data

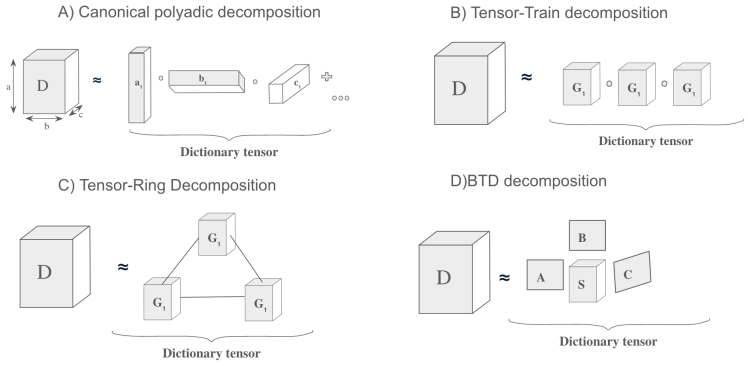


Figure 1 — CP, Tensor-Train, Tensor-Ring and BTD as dictionary tensors.

Each format represents the same patch at low rank rather than discarding its structure.

Contributions

1 — One harness

CP, Tucker, TT, TR, BTD and an unstructured baseline share the same OMP sparse coding, reconstruction and evaluation.

2 — A projection view

CP/TT/TR/BTD are one line changed inside the K-SVD atom update; Tucker is matched in-harness via separable ALS.

3 — A ranking system

Three layers, objective to opinionated: Pareto dominance, an accuracy-per-parameter scalar, profile-weighted tiers.

4 — Three regimes

Synthetic mixed-structure patches, real RGB patches and a hyperspectral cube, plus a matched-budget rate–distortion study.

The Unified Framework

What is held identical, and what is allowed to vary

$=$ Held identical

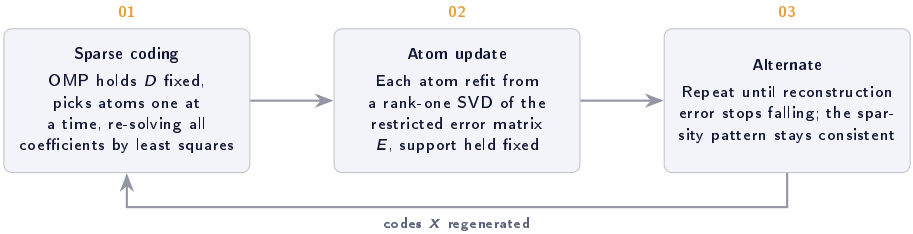
- The objective and its solver — OMP against the effective dictionary D
- Reconstruction: the map $x \mapsto Dx$
- Data, noise level, sparsity cap, seeds
- Denoising PSNR against clean held-out patches

$\neq$ Allowed to vary

- How each atom is parameterized
- How many parameters that costs to store
- The rank knob each format exposes (CP R, TT/TR r, BTD L)

Each construction usually ships with its own optimizer, so an observed gap may reflect the optimizer rather than the decomposition. Sharing the solver makes the comparison apples-to-apples.

The loop everything plugs into: K-SVD with OMP



The restricted error matrix

$$E = Y_\Omega - DX_\Omega + d_k x_{k,\Omega}^\top$$

The final term is easy to misread as *adding* error — in fact it un-subtracts atom k, leaving the gap a new d_k must fill: the error present when atom k is *absent*.

Orthogonal matching pursuit, in two mechanics

Greedy atom selection

From $r_0 = y$, $\Lambda_0 = \emptyset$, add the best-aligned atom:

$$\lambda_i = \arg\max_j |\langle d_j, r_{i-1} \rangle|, \quad \Lambda_i = \Lambda_{i-1} \cup \{\lambda_i\}$$

Residual update

The whole coefficient vector is recomputed on the active set:

$$\hat{x} = \arg\min_x \|y - D_{\Lambda_i} x\|_2, \quad r_i = y - D_{\Lambda_i} \hat{x}$$

That orthogonal projection is the defining line: the residual stays orthogonal to every selected atom, so no atom is chosen twice. The loop stops at $\|r_i\|_2 < \varepsilon$ or $|\Lambda_i| = s$.

Where a decomposition can enter

On each atom CP · TT · TR · BTD

Each atom is itself a low-parameter tensor. Atoms are independent, so the constraint is enforced atom-by-atom: *snap* the optimal unconstrained atom onto the decomposition's manifold.

$$D = A \odot B \odot C \quad \text{(Khatri–Rao)}$$

Storage falls from $n_1 n_2 n_3 K$ to $(n_1+n_2+n_3)K$.

On the whole dictionary TUCKER

Every atom is built from one shared set of per-mode vectors, so the constraint couples all atoms. The update re-fits the shared mode factors by separable ALS.

$$D = U_3 \otimes U_2 \otimes U_1 \quad \text{(Kronecker)}$$

The core plays the role of the sparse code.

The Khatri–Rao is the column-matched Kronecker: CP keeps only matched triples, Tucker keeps all $k_1 k_2 k_3$ combinations.

Tucker in the same harness

Same outer loop — OMP against $D = U_3 \otimes U_2 \otimes U_1$, giving each sample a sparse core — then update the mode factors by separable ALS. Holding the cores fixed, each U_n minimizes

$$\sum_m \|\mathcal{Y}_m - \mathcal{C}_m \times_1 U_1 \times_2 U_2 \times_3 U_3\|^2,$$

which for mode n is the linear solve

$$U_n = X_n G_n^\top \left(G_n G_n^\top + \lambda I\right)^{-1}, \quad G_m = \mathcal{C}_m \times_{p \neq n} U_p.$$

This "re-fit the shared factors" step is the Tucker analogue of the per-atom snap, and the canonical separable/Kronecker construction.

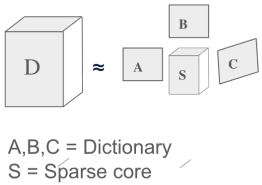


Figure 2 — the core is the sparse matrix; the three mode matrices are the dictionary.

Parameter accounting

Method	Dictionary form	Stored parameters
Vectorized	free atoms	$K \prod_i n_i$
CP (rank R)	$D = A \odot B \odot C$ (Khatri–Rao)	KRS
Tucker	$D = U_N \otimes \cdots \otimes U_1$ (Kronecker)	$\sum_i n_i k_i$
TT (rank r, N=3)	chain of cores	$K\left[r(n_1{+}n_3) + r^2 n_2\right]$
TR (rank r)	ring of cores	Kr^2S
BTD (L, b blocks)	sum of Tucker blocks	$Kb(L^N + LS)$

Table 1 — $S = \sum_i n_i$; atom shape $(n_1, \ldots, n_N)$. The sparse code (or Tucker core) is per-sample and excluded from every method equally. For Tucker the atom count is $\prod_i k_i$; only the mode factors are stored.

Every count is multiplicative in K except Tucker's, which is additive across modes — that single asymmetry drives much of what follows.

Efficiency Ranking System

Layer 1: Pareto dominance (weight-free)

On the two core axes — test accuracy and parameter-efficiency — a method is *dominated* if another is at least as good on both and strictly better on one.

No non-negative weighting can rescue a dominated method, so this layer holds regardless of anyone's priorities.

The non-dominated methods form the rate–distortion frontier; Layers 2 and 3 only rank what survives here.

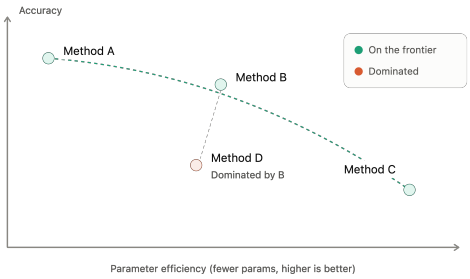


Figure 3 — Method D is dominated by B; A, B and C form the frontier.

Layers 2 and 3: a scalar, then a profile

Layer 2 — an interpretable scalar

$$\mathsf{E} = \frac{\mathrm{PSNR}}{\log_{10}(\mathrm{params})}$$

with $\mathrm{MSE} = \frac{1}{N}\sum_i (x_i - \hat{x}_i)^2$ and $\mathrm{PSNR} = 10\log_{10}(\mathrm{MAX}^2/\mathrm{MSE})$. The denominator counts *decades* of parameters, so E reads as decibels of fidelity per order of magnitude spent.

Layer 3 — profile-weighted tiers

Profile	Acc.	Par.	Speed	Rob.
Balanced	0.40	0.30	0.15	0.15
Quality (offline)	0.60	0.20	0.10	0.10
Compression (edge)	0.25	0.55	0.10	0.10
Realtime	0.30	0.20	0.40	0.10

Each axis min–max normalized to $[0, 1]$, then graded $S \geq 0.80$, $A \geq 0.65$, $B \geq 0.45$, $C \geq 0.25$, else D.

Experimental Setup

Three datasets, three different questions

Synthetic mixed-structure — $6 \times 6 \times 6$

40% separable (rank-one), 40% low-TT-rank, 20% dense, so no structure is trivially favoured. We sweep the rank directly rather than tiering one hand-picked configuration.

Real RGB patches — $8 \times 8 \times 3$

From a standard photograph, mean-removed and normalized. No ground truth, so the full three-layer ranking is applied at fixed hyperparameters.

Hyperspectral cube — $6 \times 6 \times 16$

Genuinely low-rank with one fat spectral mode. Evaluated at matched parameter budget, so we report the frontier rather than tier scores.

Dictionaries are learned on noisy training patches and evaluated by denoising PSNR against clean held-out test patches, averaged over seeds. Decomposition primitives use TensorLy.

The synthetic hyperspectral cube

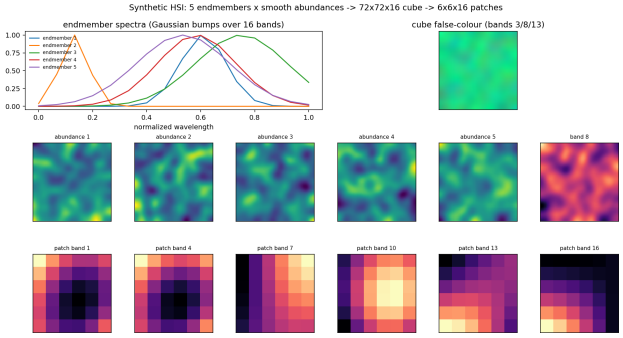


Figure 15 — five Gaussian endmember spectra mixed by smooth abundance maps into a $72 \times 72 \times 16$ cube (false-colour composite shown), cut into $6 \times 6 \times 16$ patches. Genuinely low-rank with one fat spectral mode; no real HSI capture is used.

The real RGB patches

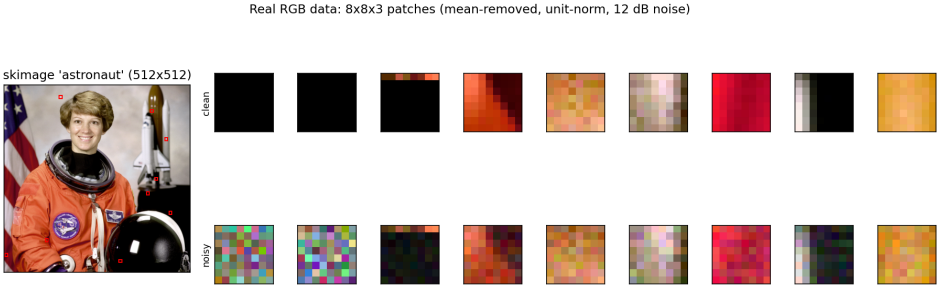


Figure 8 — the skimage astronaut image with sampled $8 \times 8 \times 3$ patch locations, and clean/noisy patch pairs.

Natural patches are smooth and approximately low-rank — the regime where structural constraints regularize rather than restrict.

Results 1: Synthetic Data

On rich, clean data the unstructured baseline wins

Model	Atoms	Params	Fit (s)	PSNR
Vectorized	120	25,920	0.85	**41.49**
CP	120	2,160	3.50	33.14
Tucker†	343	126	1.42	27.01
TT	120	5,760	1.13	38.29
TR	120	8,640	1.24	38.95

Table 3 — $6 \times 6 \times 6$, 12 dB, K=120, sparsity 6, 5 seeds. †initial out-of-harness optimizer; corrected later.

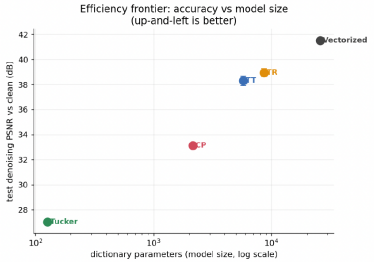


Figure 5 — up-and-left is better.

The baseline buys 41.5 dB with the largest dictionary by an order of magnitude; CP is most compact per atom but least accurate at fixed atom count.

Because 20% of the signal has no tensor structure to exploit

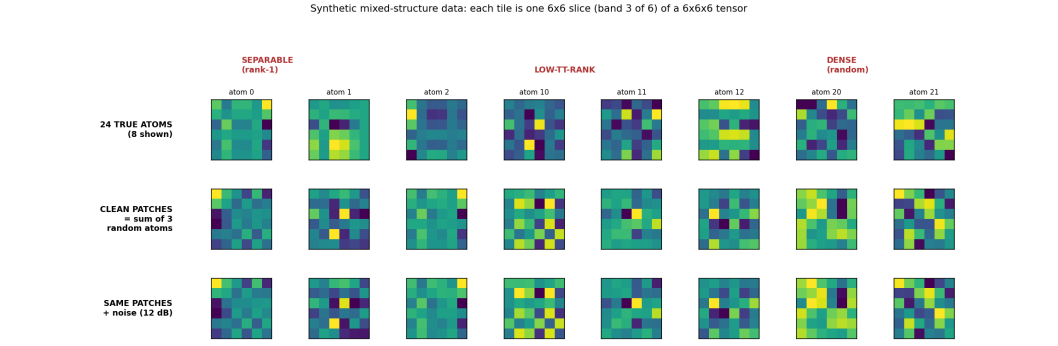


Figure 4 — eight of the 24 ground-truth atoms (separable / low-TT-rank / dense), clean patches formed as sums of three atoms, and their 12 dB noisy versions.

Any method constraining atoms to a tensor format is limited on the dense fifth of the data. At 12 dB noise imposes no strong regularization, so the baseline's extra freedom converts directly into accuracy.

Rank sensitivity: where the formats separate

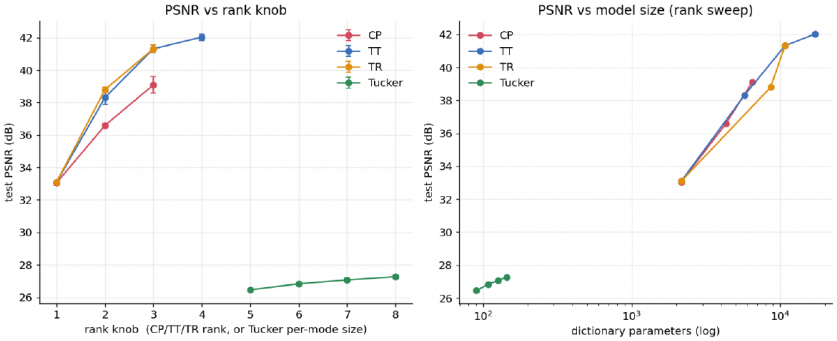


Figure 6 — left: PSNR versus each method's rank knob. Right: PSNR versus dictionary parameters under the rank sweep.

CP, TT and TR start at the identical point ($\approx 33.1\,dB$) at rank 1 — there the three decompositions coincide, each expanding into the same outer product $a \circ b \circ c$. CP then lags, sitting $\approx 2\,dB$ below TT/TR at rank 2 (36.6 vs 38.3–38.8) and needing rank 3 to reach 39 dB. TT overtakes the baseline at rank 4.

Constraint buys stability across initializations

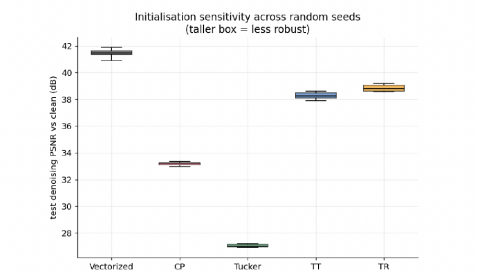


Figure 7 — taller box = more sensitive to the random seed.

- Each method is run over multiple seeds; the spread of resulting PSNR is plotted as a box.
- The baseline varies most; constrained models converge to roughly the same answer regardless of initialization.
- CP's wide bars are the fingerprint of ALS without an SVD warm start; TT and TR fit by truncation.

Seed variance is often a readout of *optimiser* quality, not *model* quality.

Results 2: Real RGB Patches

On real patches the conclusion inverts

Model	Params	Compr.	Fit (s)	PSNR
Vectorized	19,200	1.0×	0.82	30.81
CP	1,900	10.1×	2.54	**31.65**
Tucker†	153	125.5×	1.41	30.74
TT	5,400	3.6×	1.06	31.27
TR	7,600	2.5×	1.18	31.27
BTD	4,600	4.2×	3.00	31.37

Table 4 — $8 \times 8 \times 3$, 12 dB, sparsity 6. †initial out-of-harness optimizer.

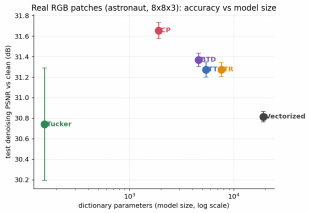


Figure 9 — structured dictionaries move above the vectorized baseline.

31.65 dB vs 30.81 dB — *the vectorized dictionary spends its extra capacity fitting noise; the structured methods cannot, so their constraint accidentally acts as a regularizer.*

CP instability: accuracy can hide a degenerate factorization

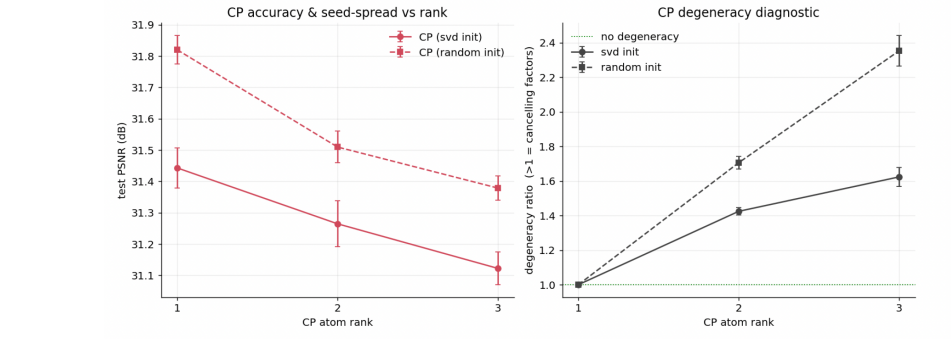


Figure 10 — left: accuracy and seed-spread versus rank for SVD vs random initialization. Right: the degeneracy diagnostic climbs with rank.

The degeneracy ratio is the sum of component magnitudes over the norm of their sum. Rank-one atoms are always non-degenerate (= 1.0); the ratio grows with rank, with random init producing markedly more degenerate, partially cancelling components than an SVD warm start (2.35 vs 1.62 at rank 3). Reconstruction can look fine while the factorization is internally degenerate — a warning if the factors are used downstream.

BTD interpolates CP and Tucker

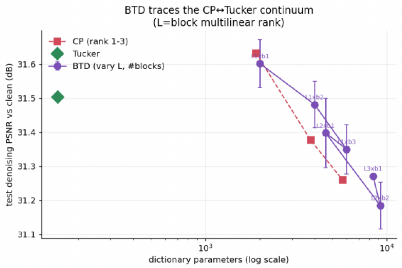


Figure 11 — BTD traces the CP ↔ Tucker continuum as block rank L and block count vary.

The L=1, one-block configuration lands exactly on CP rank-one, confirming the endpoint identity, and the family traces the continuum toward Tucker. In this noisy regime the most constrained point is best — extra capacity fits noise.

Tucker stays almost stationary because its parameter count grows far more slowly than its dictionary size: unlike CP and BTD, which add parameters per atom, Tucker stores only three shared mode dictionaries. Parameters grow roughly linearly with mode dimension while the atom count grows multiplicatively.

Refinement of the Tucker Optimizer

An optimizer bug, not a structural verdict

The original Tucker-DL failed on an **objective mismatch**: it learned its three mode dictionaries in isolation,

$$Y_{(1)} \approx U_1 X_1, \qquad Y_{(2)} \approx U_2 X_2, \qquad Y_{(3)} \approx U_3 X_3,$$

but a Tucker dictionary is never seen alone — only as $D = U_3 \otimes U_2 \otimes U_1$. The fix keeps OMP against the Kronecker dictionary and updates by separable ALS: solve U_1 with (U_2, U_3) fixed, then U_2, then U_3, each an *exact closed-form least-squares* block.

RGB patches

$30.66\ dB \longrightarrow 32.16 \pm 0.09\ dB$

at an identical 153-parameter budget

Synthetic data

$27.01\ dB \longrightarrow 29.45 \pm 0.69\ dB$

a gain of approximately $+1.5\ dB$

A fairness correction to one optimizer moved Tucker from last place to a Pareto-dominant winner at identical size.

Efficiency ranking with the corrected Tucker: RGB

Model	PSNR	Params	E	Par.	Tier	Bal.
Tucker	32.16	153	14.72	yes	S	0.892
CP	31.71	1,900	9.67	no	C	0.429
TT	31.28	5,400	8.38	no	C	0.389
BTD	31.38	4,600	8.57	no	C	0.385
TR	31.31	7,600	8.07	no	C	0.306
Vectorized	30.80	19,200	7.19	no	D	0.230

Table 5 — corrected Tucker, 5 seeds, $E = \mathrm{PSNR}/\log_{10}(\mathrm{params})$. **Caveat:** Tucker's $9 \times 9 \times 3$ grid yields 243 implicit atoms for 153 explicit parameters, where other methods get ≈100 atoms at the same count — part of its edge is extra atoms at the same sparsity level.

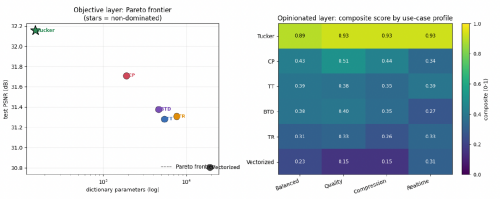


Figure 12 — left: objective Pareto layer (stars are non-dominated); only Tucker is on the frontier. Right: profile-weighted composite scores.

Efficiency ranking with the corrected Tucker: synthetic

Model	PSNR	Params	E	Tier	Bal.
Vectorized	41.25	25,920	9.35	A	0.700
TR	36.98	8,640	9.39	C	0.413
TT	36.46	5,760	9.70	B	0.482
CP	32.65	2,160	9.79	C	0.417
Tucker	29.45	126	14.02	C	0.416

Table 6 — in-harness Tucker, 5 seeds. Every method is Pareto-optimal: the frontier is monotone, so the objective layer prunes nothing.

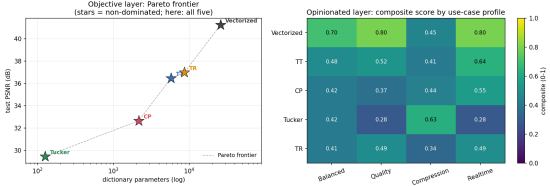


Figure 13 — with a monotone frontier, all the discriminating work falls to the opinionated layers.

Results 3: Matched-Budget Hyperspectral

Matched parameter budget removes the hand-picked ranks

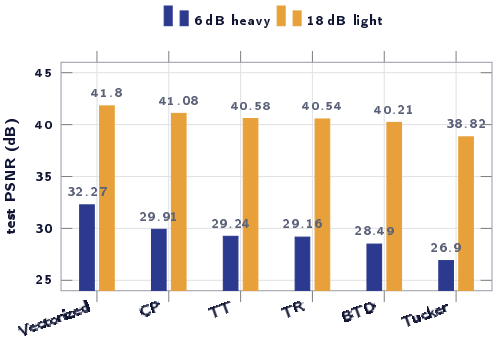


Table 7 — HSI cube $6 \times 6 \times 16$ at the $\approx$5,000-parameter slice. Tucker operates at 304 parameters: it cannot reach the budget without exceeding a reasonable atom cap.

A fat mode is not a high order

Giving Tensor-Train a large third mode did *not* hand it the win. TT's advantage is high tensor *order*, not one wide mode.

The ranking does not flip with noise

Between heavy (6 dB) and light (18 dB) noise only the absolute PSNR shifts; the ordering is stable.

Less is more on a low-rank cube

Accuracy falls as the budget grows. The baseline leads at 5,000 parameters partly because its atoms cost 576 each.

Why the matched-budget study is restricted to HSI

- **Isolating the wide mode.** The experiment tests whether Tensor-Train benefits from a single wide mode or from higher-order structure. The $6 \times 6 \times 16$ HSI tensor is the only dataset in the benchmark that provides a wider mode in isolation.
- **Selection bias needs a validation split.** Choosing ranks on training reconstruction error would unfairly favour larger, less regularized models. HSI is split into training, validation and test: each configuration trains on the training set, the rank is chosen on validation, and the selected model is evaluated *once* on test.
- **Scope.** These considerations justify restricting the analysis to HSI, but the conclusions apply to this experimental setting rather than universally.

Rate–distortion sweep across noise levels

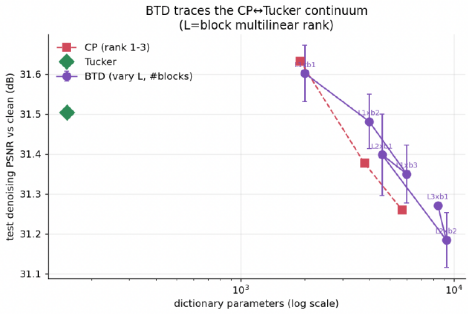


Figure 14 as printed in the manuscript.

Note: the embedded image duplicates Figure 11 (the BTD continuum). The caption describes the matched-budget rate–distortion curve at 6 dB and 18 dB, whose numbers appear on the previous slide — worth regenerating before submission.

Discussion

Three themes

(i) Regime dependence

There is no absolute best decomposition. The baseline wins on rich clean synthetic data; structured dictionaries win on noisy real patches where their constraint regularizes; at matched budget on a low-rank cube, few full atoms or a tiny Tucker are most efficient. Which to use is a function of signal order, noise and intrinsic rank — not a fixed ranking.

(ii) Confounds can dominate structure

The Tucker rebuild moved it from last to first at identical size purely by fixing the optimizer, and the matched-budget study revised the earlier "structured wins" story by revealing that some of that advantage was really the freedom to buy many cheap atoms.

(iii) Matched-parameter is not matched-atom

At equal storage the baseline is forced into few heavily regularized atoms while structured methods spend the same budget on many. Which budget is "fair" depends on whether you care about storage or per-atom quality.

Limitations

Implementations, not canonical algorithms

CP/TT/TR/BTD here are projection variants inside a shared K-SVD loop — not jointly optimized factors, PALM-based TT-DL or SeDiL. The numbers rank these implementations, not those methods definitively.

Modest data, seed-only error bars

Hundreds of patches, single images, a few noise levels; the bars capture seed variance alone, and the HSI cube is synthetic.

The budget study caps atom counts

Capping for tractability bounds Tucker artificially — it is why Tucker sits at the far left of the rate–distortion curve rather than on it.

The speed axis understates tensor methods

The harness materializes the full effective dictionary and runs identical OMP for fairness, discarding the fast structured transforms that are a genuine advantage of Kronecker and TT dictionaries.

A fuller treatment would report several implementations per structure and show the spread — natural next steps rather than flaws to hide.

One harness, six dictionaries,
and a ranking that can be re-derived anywhere

Conclusion

Connected

Five tensor-structured dictionaries and an unstructured baseline differing only in how each atom is parameterized — linked to DL through the Khatri–Rao and Kronecker forms.

Measured

Across synthetic, real and hyperspectral data the best decomposition is regime-dependent, and implementation confounds can outweigh structural differences.

Extensible

A matched parameter budget both tightens and, in places, reverses naive conclusions. A new decomposition is one projector and one parameter rule away.

References i

- M. Aharon, M. Elad, and A. Bruckstein. K-SVD: An algorithm for designing overcomplete dictionaries for sparse representation. *IEEE Trans. Signal Processing*, 2006.
- Y. C. Pati, R. Rezaiifar, and P. S. Krishnaprasad. Orthogonal matching pursuit. *Asilomar Conf. on Signals, Systems and Computers*, 1993.
- T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 2009.
- L. R. Tucker. Some mathematical notes on three-mode factor analysis. *Psychometrika*, 1966.
- I. V. Oseledets. Tensor-train decomposition. *SIAM J. Scientific Computing*, 2011.
- Q. Zhao, G. Zhou, S. Xie, L. Zhang, and A. Cichocki. Tensor ring decomposition. *arXiv:1606.05535*, 2016.
- L. De Lathauwer. Decompositions of a higher-order tensor in block terms — Part II. *SIAM J. Matrix Analysis and Applications*, 2008.

References ii

S. Hawe, M. Seibert, and M. Kleinsteuber. Separable dictionary learning. *CVPR*, 2013.

C. F. Caiafa and A. Cichocki. Computing sparse representations of multidimensional signals using Kronecker bases. *Neural Computation*, 2013.

J. Kossaifi, Y. Panagakis, A. Anandkumar, and M. Pantic. TensorLy: Tensor learning in Python. *JMLR*, 2019.